TraceBack — Performance and Load Test Report

6 Nov 2026 · companion to the Test, Concurrency and Security reports

Before the changes, the API locked up at 25 users using the site at once and answered nothing until the rush was over. After six changes, it served 100 users flat out with no errors at about 250 requests a second (3.4 times as many), and 200 people browsing normally with 95% of requests under 0.85 s. A page view now sends 0.8 MB instead of 18 MB.

TraceBack is the IIT Kanpur Lost & Found portal. All load tests ran on a copy of the system in an isolated test environment, never on the live site. The changes are in the team repository (commit `perf: keep the API responsive under load…` on branch `d3/integration`) and reach the server with `deploy/upload.ps1`.

Measure	Before	After
Most users served flat out with no errors	10	100 (the most tested)
Users browsing normally with 95% of requests under 1 s	20	200
Requests answered per second, 5–100 users flat out	71–74	231–255
Median response, 10 users flat out	467 ms	91 ms
Data per page view	18–21 MB	0.7–1.0 MB
Server CPU per page view	0.14–0.18 CPU-seconds	about 0.055 CPU-seconds
50-user rush for 30 s	77% of requests failed; answered again 27 s after it ended	no failures; no recovery needed
Functional tests	284 passed	459 passed (7 of them for these changes), system tests 15 of 15
Concurrency checks	7 of 14	14 of 14 (with the concurrency fixes)

1. Scope and method

- **Machine:** the workspace has 2 CPU cores, the same as the t3.small, but 8 GB of memory instead of 2 GB and no real network. The website, API, worker, PostgreSQL and the load generator all ran on it. The load generator used 5–23% of one core, so the server had a little less CPU than on AWS.
- **Data:** a year's worth: 5,000 students, 5 guards, 10,000 items (1,700 held at offices, 1,500 open lost reports), 3,200 photos and 1,500 claims. Every photo is a 12-megapixel, 2.9 MB JPEG, the size a phone camera makes.
- **Virtual users** act like a browser. Each page view is one of: the Found page (30%), search or filter (18%), an item page (22%), the Lost page (12%), My reports and claims (10%), report a lost item with a photo (4%), claim an item (2%), Returned items (2%). A list page loads its 12 card photos, 6 at a time. One user in 25 is a guard opening the staff pages. Nothing is cached, as on a first visit.
- **Tests:** *ramp* = 1 to 100 users with no pause between pages (the limit); *campus* = 10 to 400 users pausing about 5 s between pages (normal use); *spike* = 50 users at once for 30 s, then how long until the API answers again. Every step starts on a fresh database and a freshly started API. A request that takes over 15 s counts as timed out.
- **Measures:** requests answered per second; median (p50) and 95th-percentile (p95) response time; errors; CPU and memory of each part; bytes sent. A profiler (py-spy) and a query counter showed where the time went.
- **Tools:** our own load generator (`testing/perf/loadgen.py`, Python and aiohttp) and psutil. Everything is in `testing/perf`, with a README.

2. What we found

F1 — Critical: the API locks up at 25 users. In the ramp test with 25, 50 and 100 users, not one request succeeded, and in normal use with 50 to 400 users 99.4–100% failed. The API was not busy: its CPU fell to about 1%. The profiler showed why. FastAPI runs each request on one of 40 worker threads, in several steps, and the database pool had 15 connections. Requests that already held a connection needed another thread for their next step, while all 40 threads waited for a free connection, so nothing moved until 30-second timeouts released them. During the 50-user rush, 77% of requests failed, and the API answered again 27 s after the rush ended. While locked, it also ignored the stop signal and had to be killed. Docker does not restart a process that hangs without crashing. The same lock-up is why 100 users browsing at once (CON-14) failed.

F2 — High: list pages send full-size photos. Photos were 77% of requests and 99.97% of the bytes. Each card on the Found page showed the original 2.9 MB photo, so that page alone sent about 34 MB, and the average page view 18–21 MB. On the t3.small, that is less

than one page view a second within its baseline network speed, and about 5,500 page views a month within AWS's 100 GB of free data transfer. Serving a 2.9 MB file through Python also kept the API process busy.

F3 — Medium: one process uses one CPU core. The API sat at 92–95% of one core while the machine as a whole was only about 65% busy. A single Python process can only use one core.

F4 — Medium: uploads block everyone. The photo and proof upload endpoints were `async def` but call the database, which waits. That ran on the API's single event loop, so every other request waited during an upload's database work.

F5 — Low: more database queries than needed. Every signed-in request used 2 queries to find the session and its user; a photo used 5; the staff claims page used 21 for 20 claims, one more for each claim's category and office.

F6 — Low: the AI queue also runs inside the API. `AI_WORKER_IN_API` defaults to true and `docker-compose.yml` does not turn it off, so the AI model is loaded both in the API and in the worker container, and AI work competes with requests. This was not measured with the real model, which can't be downloaded here; the tests used the small built-in one.

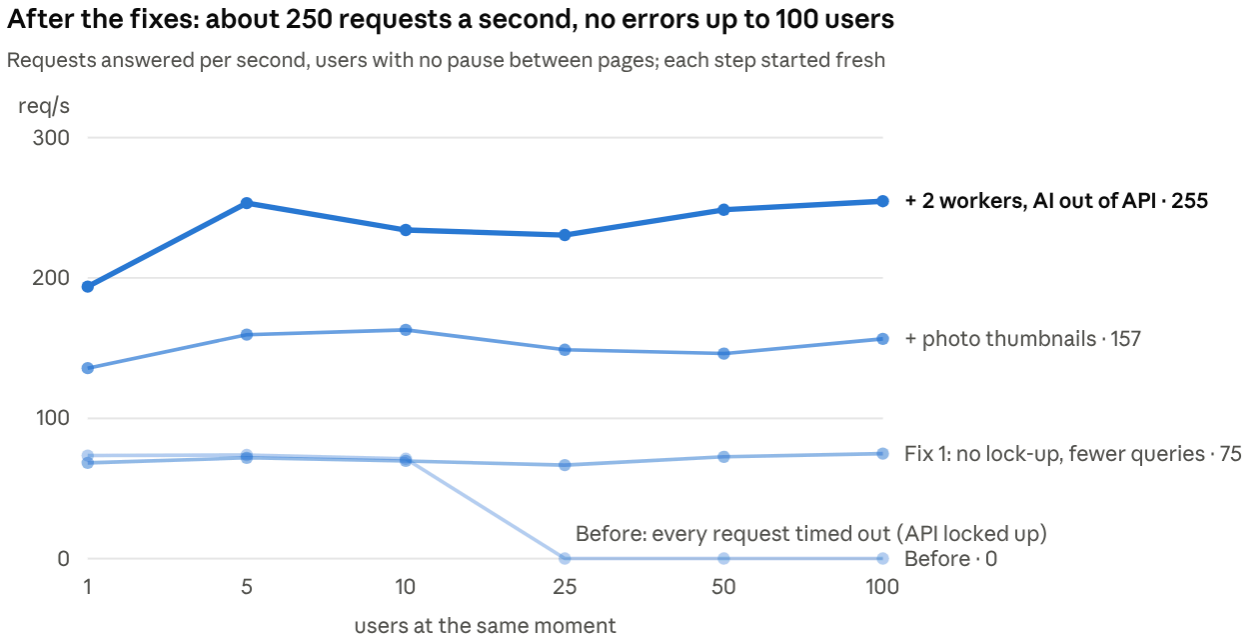
3. What we changed

#	Change	Fixes	Files
1	At most 30 requests are worked on at once in each API process, and the database pool has a connection for each of them (10 kept open, up to 35). Extra requests wait their turn without holding a thread or a connection; after 20 s they get a 503 “server busy”.	F1	`core/capacity.py` (new), `core/database.py`, `core/config.py`, `main.py`

#	Change	Fixes	Files
2	Each item photo gets a 640-pixel WebP copy (about 30–60 KB), made at upload; older photos get one the first time they are shown. Lists, the staff desk, claim pages and the item page's photo strip use it; the large photo on the item page stays full size. Thumbnails carry no EXIF data, so no GPS location, and images over 60 megapixels are refused.	F2; also stops the image bomb (ADV-18)	`services/storage.py`, `routers/items.py`, `services/items.py`, `frontend/lib/api.js` and 7 pages and components
3	Two API worker processes, one per CPU core	F3	`backend/Dockerfile`
4	The upload endpoints are plain `def`, so they run in worker threads. The item or claim row is locked while photos and proofs are counted, so parallel uploads can't pass the 3-photo limit (CON-10).	F4	`routers/items.py`, `routers/claims.py`, `services/items.py`, `services/claims.py`
5	One query for the session and its user; a photo needs 2 queries instead of 5; the staff claims page needs 9 instead of 21	F5	`services/auth.py`, `routers/items.py`, `services/claims.py`
6	The AI queue runs only in the worker container (`AI_WORKER_IN_API=false`); the API uses uvloop and httptools, faster parts for its web server	F6	`deploy/docker-compose.yml`, `requirements.txt`

The AI worker also decodes photos at reduced size before sending them to the AI (`services/ai/vision.py`). Seven new tests cover thumbnails, the pixel limit, the single session query and the request limit (`tests/test_performance_changes.py`).

4. Results

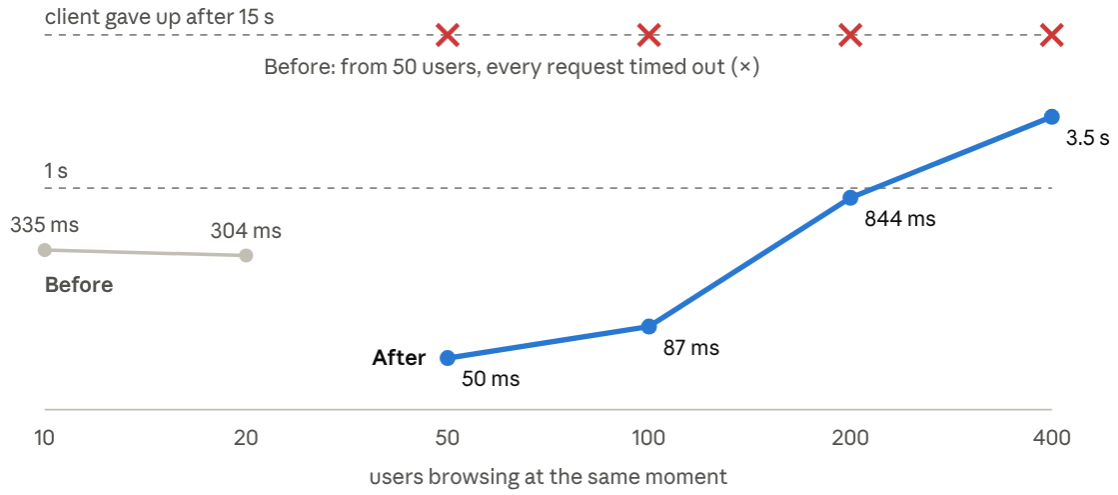


testing/perf/results: baseline_ramp, s1_code_ramp, s2_thumbs_ramp, final_ramp · 30 s per step after 5 s warm-up

Each stage was measured on its own. Fix 1 alone ended the lock-up, with no errors at any step, but throughput stayed near 70 requests a second because the API was still sending full-size photos. Thumbnails doubled it to 146–163. Two workers, uvloop and moving the AI queue out of the API took it to 231–255, where the machine's two cores were 87–90% busy. Before thumbnails existed for the seeded photos, the first views made them on the spot; that one-off cost held throughput at 83–91 requests a second.

Normal use: after the fixes, 200 users browse with 95% of requests under 0.85 s

Slowest 5% of requests (p95, log scale); users pause about 5 s between pages

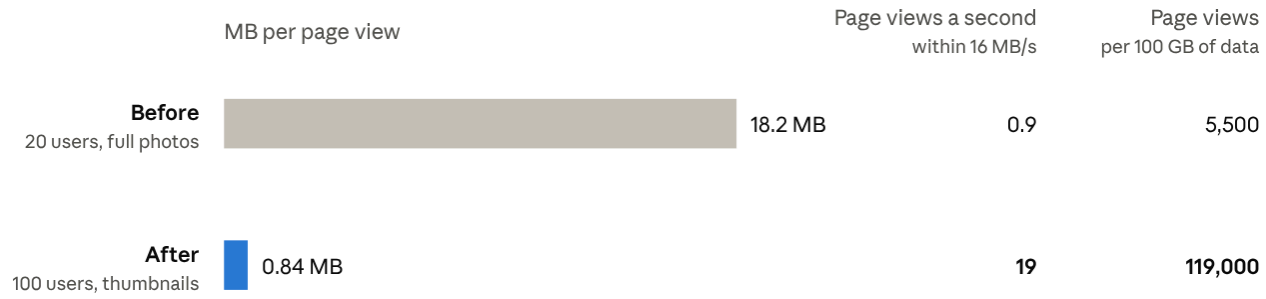


testing/perf/results: baseline_campus_small (10, 20 users), baseline_campus and final_campus (50–400 users) · 45 s per step

With normal pauses, the old version stayed fast up to 20 users and locked up at 50. The new one kept 95% of requests under 0.1 s at 50 and 100 users and under 0.85 s at 200. At 400 users it was overloaded but kept working: p95 3.5 s, with 4 failed requests out of 10,766 (0.04%). All 7 failures across the 200- and 400-user steps were connection errors seen by the load generator; the API returned no error status.

Photo thumbnails cut the data per page view from 18 MB to 0.8 MB

Average data sent per page view in the normal-use test, and what that allows on the t3.small



Estimates from the measured bytes: 16 MB/s is the t3.small sustained network baseline, 100 GB is AWS free data transfer a month.

testing/perf/results: baseline_campus_small (20 users) and final_campus (100 users), mb_per_page_view · limits from AWS and a published benchmark (Sources)

Request (10 users, flat out)	Median before	Median after
Found page list	240 ms	89 ms

Request (10 users, flat out)	Median before	Median after
Search or filter	274 ms	85 ms
Item page	218 ms	75 ms
My claims	301 ms	100 ms
Card photo	497 ms (2.9 MB)	97 ms (55 KB thumbnail)
Report a lost item	278 ms	112 ms

Spike: 50 users at once for 30 s now get no errors, with p95 0.73 s, and the API answers straight away when the rush ends.

Nothing else broke. All 459 unit and integration tests and all 15 browser system tests pass, and the Found page shows the thumbnails. In the concurrency tests, CON-14 (100 users browsing) now passes with no errors; before it had 52 errors and 11 timeouts. CON-10 still holds the 3-photo limit, thanks to the new row lock. With the row locks in claims and custody as well, all 14 concurrency checks pass (see the Concurrency tab).

5. Costs and trade-offs

- **Uploads are slower.** The median photo upload went from about 36 ms to about 290 ms, because the thumbnail is made during the upload. Uploads are about 2% of page views.
- **More memory.** The two API processes used 0.40–0.55 GB together in the tests, against 0.12–0.17 GB for the single old process. The t3.small has 2 GB, shared with PostgreSQL, the website and the worker with its AI models. After deploying, check `sudo docker stats`; if memory runs short, go back to one worker. Changes 1, 2, 4, 5 and 6 don't depend on it.
- **CPU credits.** A t3.small earns 20% of each core as its sustained baseline; above that it spends credits, and by default AWS then charges $0.05 per vCPU-hour. Each page view now needs about a third of the CPU, so the same traffic uses far fewer credits. Our estimate: about 7 page views a second fit within the baseline, against about 2.5 before.
- **"Server busy" is possible.** Past 30 requests at once per process, extra requests wait, and after 20 s get a 503. That never happened in any test.

6. Limits of this test

- **Not the real server.** The workspace has 4 times the memory and no network between the browser and the server, and it doesn't use CPU credits. The t3.small figures for network and data are calculations from the measured bytes, not measurements.
- **Caddy was not in front.** HTTPS and gzip add a little work on the real server. The Next.js website was not the bottleneck: it used under 1% CPU, because its pages are built in advance.
- **Pessimistic on photos.** No browser cache, and the test photo is noisy, so its thumbnail (55 KB) is larger than most real ones will be.
- **Small AI model.** The real AI model was not loaded (see F6).

7. Next steps

1. Deploy with `deploy/upload.ps1`, then check `sudo docker stats` and open a Found page.
2. Never point the load tests at the live site. To measure the real server, watch `docker stats` and the response times while people use it.
3. Ship these changes in the same release as the concurrency, security and recovery fixes (see those reports): they share files and were tested together.

Sources: Amazon EC2 T3 instances (baseline 20% per vCPU, Unlimited mode by default, $0.05 per vCPU-hour) · EC2 Network Performance of t3 (t3.small baseline about 0.13 Gbit/s, a benchmark) · AWS Free Tier data transfer expansion (100 GB a month)